

CQRS

Command Query Responsibility Segregation — шпаргалка

Backend Cheat Sheet

СУТЬ ПАТТЕРНА

Вместо единой **CRUD-модели** — две специализированные:

✦ Write Model (Commands)

- Бизнес-логика и валидация
- Работа с агрегатами (DDD)
- Строгая консистентность
- ACID транзакции

✦ Read Model (Queries)

- Скорость и гибкость
- Денормализованные проекции
- Пересекают границы агрегатов
- Минимум бизнес-логики

Ключевая идея: операции чтения и записи живут в разных мирах — у них разные требования, разные модели, разные стратегии.

КОГДА ПРИМЕНЯТЬ / НЕ ПРИМЕНЯТЬ

✓ ПРИМЕНЯЙ, ЕСЛИ:

- ✓ Репозитории раздулись до 50+ методов
- ✓ Запросы с 5+ JOIN'ами тормозят приложение
- ✓ 90%+ трафика — чтение (асимметрия нагрузки)
- ✓ Сущность отображается по-разному в разных контекстах
- ✓ Нужны специализированные БД (Elasticsearch, ClickHouse)

✗ НЕ ПРИМЕНЯЙ, ЕСЛИ:

- ✗ Простое CRUD-приложение без роста сложности
- ✗ Команда мала и не готова к eventually consistency
- ✗ Добавляешь «на вырост» без реальной проблемы

Правило: начинать с простого, усложняя по необходимости.

ТРИ ВАРИАНТА РЕАЛИЗАЦИИ

Вариант	Архитектура	Сложность	Консистентность	Перформанс	Когда использовать
1. Уровень приложения	1 БД, 2 набора классов/репозитория	Низкая	Строгая	Умеренный	Проблема в сложности кода, репозитории раздулись
2. Две схемы в БД	1 БД (write_schema + read_schema)	Средняя	Sync/Async	Высокий	Чтение тормозит, нужна денормализация, JOIN'ов много
3. Отдельные БД	PostgreSQL (write) + Elasticsearch/ClickHouse (read)	Высокая	Eventually	Очень высокий	Радикально разный профиль нагрузки, нужны спец. БД

СИНХРОНИЗАЦИЯ МОДЕЛЕЙ

Синхронная (в транзакции)

- ✓ Строгая консистентность, данные сразу актуальны
- ✗ Замедляет запись
- ✗ Ошибка проекции = откат всей транзакции

Асинхронная (очередь / фоновый воркер)

Write DB → Kafka / RabbitMQ → Projector → Read DB

- ✓ Запись не замедляется
- ✓ Независимое масштабирование
- ✓ Несколько подписчиков на одно событие
- ✗ Eventually Consistent (задержка 1–60 с)
- ✗ Нужен мониторинг lag'а проектора

Паттерн Transactional Outbox — для гарантии доставки событий при асинхронной синхронизации.

COMMANDS & QUERIES (ОПЦИОНАЛЬНО)

Ортогональный паттерн, хорошо сочетается с CQRS, но не обязателен.

```
// Command
class CreateProductCommand {
    Name: string; Price: decimal;
}
// Command Handler
class CreateProductHandler {
    Handle(cmd) → await repo.Save(product);
}
// Query Handler
class GetCatalogHandler {
    Handle(q) → await readRepo.GetCatalog();
}
```

ПРЕИМУЩЕСТВА HANDLERS:

- ✓ Single Responsibility — 1 handler = 1 операция
- ✓ Легко добавить Logging/Caching через декораторы
- ✓ Минимальные зависимости в каждом классе

ЧАСТЫЕ ВОПРОСЫ

Может ли Command возвращать значение?

- ❌ `Product CreateProduct(...)` — возвращать весь агрегат ❌
- ✅ `Guid CreateProduct(...)` — вернуть только ID ✅
- ✅ `CommandResult CreateProduct(...)` — статус + ID/ошибки ✅

Query с побочным эффектом (счётчик просмотров)?

- Query возвращает данные (чистый, без side effects)
- Асинхронно публикует событие `ProductViewedEvent`
- Отдельный handler обновляет аналитику

Куда отнести парсинг файла?

Это обычная сервисная операция — не Command и не Query. CQRS = про чтение/запись в хранилище, а не классификацию любой операции.

ТИПИЧНЫЕ ОШИБКИ И ПОДВОДНЫЕ КАМНИ

⚠ Eventually Consistency — скажи пользователю!

Не делай вид, что система синхронная. Используй честный UX:

«Изменения будут опубликованы в течение ~30 секунд»

Или оптимистичное обновление на клиенте с пометкой «Синхронизация...»

⚠ Лишняя денормализация Read Model

Каждое поле в read-модели = ответственность за синхронизацию. Одно изменение может затронуть десятки витрин. Следи за lag'ом проектора.

⚠ CQRS везде = антипаттерн

CQRS — инструмент управления сложностью. Нет сложности → не нужен CQRS. Добавление «на всякий случай» только усложнит проект.

РЕАЛЬНЫЕ КЕЙСЫ

Е-commerce: каталог 100K товаров (Вариант 2)

Проблема: запрос с 4 JOIN'ами занимал 800ms

Решение: денормализованная `product_catalog_view` в `read_schema` с ценой, рейтингом, остатками — считаются заранее

Синхронизация: Background Worker по событиям `PriceChanged`, `ReviewAdded`

Результат: 800ms → 100ms (×8)

Финтех: аналитика 50M транзакций (Вариант 3)

Проблема: агрегация за 3 месяца в PostgreSQL — 2 минуты, CPU в космосе

Решение: PostgreSQL (write) + ClickHouse (read) через Kafka

Задержка: ~10 секунд (приемлемо для бизнес-аналитики)

Результат: 2 мин → 2 сек (×60)

ЭВОЛЮЦИЯ ПРОЕКТА — НЕ РЕВОЛЮЦИЯ

